

Project Executable Files

Date	10 July 2026
Team ID	xxxxxx
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	3 Marks

Project Executable Files

This section contains all executable, deployable, and supporting files required to run the Rising Waters: A Machine Learning Approach to Flood Prediction system. The project includes the complete source code, machine learning models, datasets, Flask web application, configuration files, and documentation. The provided files enable evaluators to execute, test, and deploy the application successfully.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	No (Not Required)
6	Deployed application URL (if hosted)	No (<i>Update if deployed</i>)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	No (<i>Add after recording</i>)

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```

Rising-Waters-Flood-Prediction/
├── app.py                - Flask application entry point
├── requirements.txt       - Python dependencies
├── runtime.txt           - Python runtime version
├── Procfile              - Deployment configuration
├── README.md             - Project documentation and setup guide
├── dataset/              - Historical datasets
│   ├── flood dataset.xlsx
│   └── rainfall in india 1901-2015.xlsx
├── models/               - Trained machine learning models
│   ├── best_model.pkl
│   ├── scaler.pkl
│   ├── feature_columns.pkl
│   └── feature_stats.json
├── notebooks/            - Data analysis and model training
│   └── eda_and_modeling.ipynb
├── templates/            - HTML templates
│   ├── home.html
│   ├── predict.html
│   ├── result.html
│   ├── flood.html
│   ├── no_flood.html
│   └── layout.html
├── static/               - Static resources
│   ├── css/
│   ├── js/
│   └── images/
├── utils/                - Utility modules
│   └── explainers.py
└── __pycache__/          - Python cache files
  
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://rising-waters-flood-prediction-82wf.onrender.com/
Login Credentials (Demo)	Not Required (No authentication implemented)
Platform / Hosting Provider	Render

Repository Link	https://github.com/Harika264/Rising-Waters-Flood-Prediction-Aiml-track
Demo Video Link	https://drive.google.com/file/d/1tk7fsVT4PyTaH8thRqOiUK5BSxnPnBfc/view?usp=drive_link

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Prerequisites

- Python 3.10 or above
- pip
- Virtual environment (recommended)

Steps

1. Clone the GitHub repository.
2. Open the project folder in VS Code or Terminal.
3. Install the required dependencies:

```
pip install -r requirements.txt
```
4. Run the Flask application:

```
python app.py
```
5. Open the application in your browser:

```
http://127.0.0.1:5000
```
6. Enter rainfall and weather parameters.
7. Click **Predict** to view the flood prediction result.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Prediction accuracy depends on the quality and completeness of the historical dataset.	Improve dataset size and quality by adding more recent weather records.
2	The current application uses historical data and does not fetch live weather information.	Future enhancement: integrate real-time Weather APIs (e.g., OpenWeatherMap or IMD).
3	The system predicts flood risk for provided inputs but does not display flood locations on a map.	Future enhancement: integrate GIS mapping and location-based visualization.